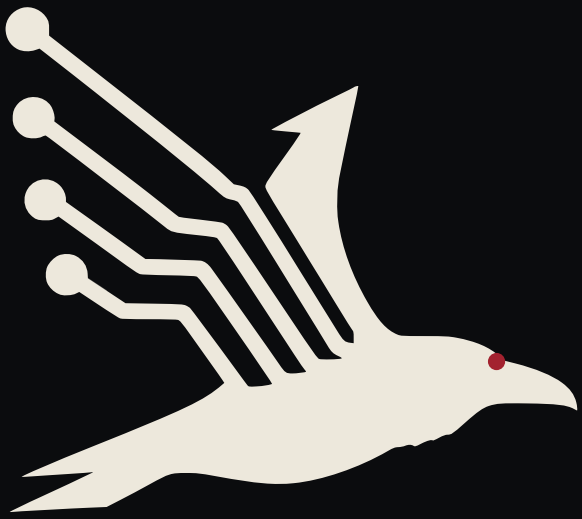


Ravenhelm Release Operations

Methodology

A TIERED METHODOLOGY FOR ARTIFACT CUSTODY & MULTI-SURFACE DEPLOYMENT

Nathan Walker

RAVENHELM LLC · 2026

Ravenhelm Release Operations Methodology

“Builds are the unit; releases are names.
Releases never rebuild code.”

— THE CORE AXIOM (ADR 0007)

Chapter 1: What This Is and Who It's For

The Release Crisis

Modern software engineering teams have largely solved issue tracking and local development loops, but their release pipelines remain profoundly fragile. Consider the standard modern deployment ritual:

An engineer opens a pull request. Automated CI runs linting, unit tests, and integration suites. Peer review passes, and the branch merges to `main`. At this point, the code is considered "ready." To ship it, an operator triggers a release workflow or pushes a git tag (`v1.4.0`).

The release workflow begins by checking out the code from scratch. It spins up a fresh runner, downloads container base images, pulls downstream dependencies from external package registries, runs the build command, packages a container, and pushes it to production.

In this ritual lies a critical vulnerability: **the artifact that runs in production was never tested in staging.**

Even if the source commit is byte-identical, the build environment is not. An upstream dependency published a patch release twenty minutes ago. A network mirror resolved a different digest for a Debian base image. An ambient timestamp changed a bundle hash. The build that passed QA was discarded; an unverified twin was compiled in its place and shipped directly to production users.

When an incident inevitably strikes, the team faces compounding failures: 1. **The Rollback Illusion:** The team attempts to rollback by triggering a redeploy of the previous git tag. But because the pipeline rebuilds on deploy, the previous tag also rebuilds from scratch against current external mirrors, frequently failing to compile or producing a completely different defect. 2. **Surface Drift:** The company's backend service was updated, but the developer CLI tool or internal administrative TUI was left on an older version. Within hours, team members corrupt cluster state because their local client tools send deprecated schema payloads.

3. **The Cutover Trap:** Under the guise of a routine deployment, an engineer merges changes that silently alter a database engine, swap a message bus transport, or deprecate an API authorization model without a migration runway. The deployment succeeds, but the system suffers an outage because the team confused a routine code deployment with an architectural cutover.

The **Ravenhelm Release Operations Methodology (RROM)** is a systematic operating discipline designed to eliminate these failure modes permanently.

Who This Is For

RROM is authored for the **Release Operator, Platform Engineer, and Technical Lead** responsible for the sovereign delivery of production software.

It is designed for organizations that reject "ambient magic" and demand absolute artifact custody. Whether you are running a single-operator startup delivering across Docker and Homebrew, or an enterprise orchestrating Kubernetes clusters and client-side applications, RROM provides an authoritative, installable framework.

The Delivery Contract

RROM establishes an unambiguous contract between the development stream and the runtime surface: - **Build once on ingress:** No artifact is ever compiled twice. - **Promote by reference:** Environments change what they run by repointing immutable pointers to proven digests. - **Lockstep across surfaces:** Backends, CLIs, and client applications move through release gates in synchronized lockstep. - **Fail-closed verification:** A deployment is not complete when a pipeline returns exit code `0`; it is complete only when verified running against live network probes.

How to Read This Book

- **For Immediate Implementation:** Read Chapter 2 (The Core Axiom), Chapter 3 (The Numbered Principles), Chapter 6 (The Three Physical Rails), and Chapter 8 (The Tiered Adoption Ladder).
- **For Architectural Governance:** Read Chapter 9 (Cutovers vs. Deployments) and Chapter 11 (Security Blast Radius and Elevation).
- **For Tool & Package Maintainers:** Read Appendix A (Formula Inreplace Playbook) and Appendix B (The Vitki Reference Specification).

Chapter 2: The Core Axiom

The Axiom Stated

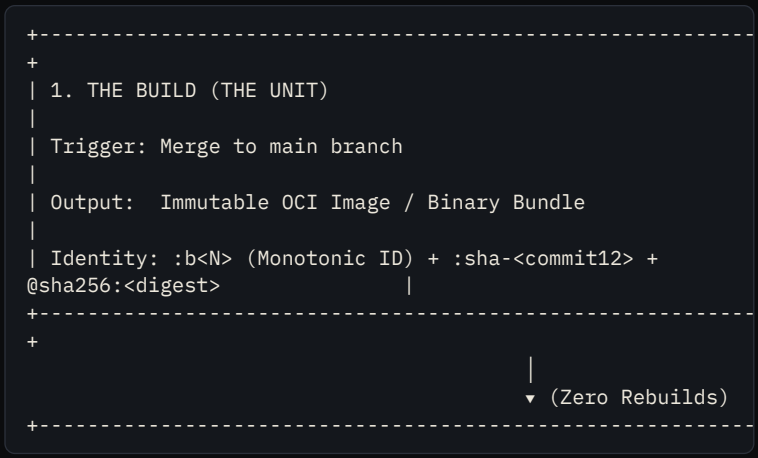
At the heart of the Ravenhelm Release Operations Methodology lies a single governing law:

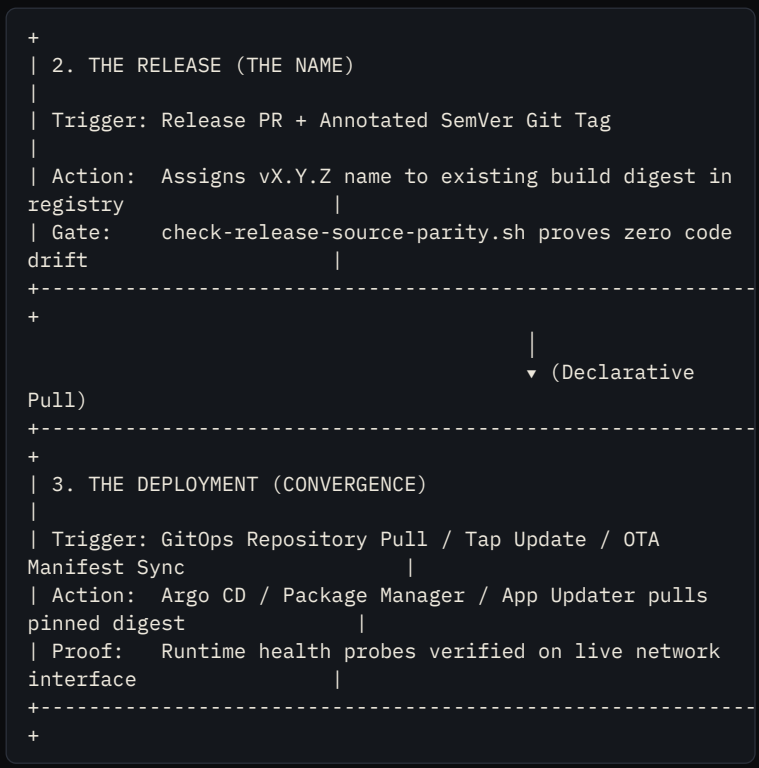
> ***Builds are the unit; releases are names.***
> A build is an immutable artifact produced once. A release is an authorized name assigned to an existing build. A deployment is the declarative reconciliation of a target environment to that named artifact.
>
> ***Releases never rebuild code.***

This axiom, formalized in estate doctrine under **ADR 0007** (with the Vitki reference implementation), fundamentally restructures the relationship between version control, artifact storage, and runtime orchestration.

Deconstructing the Three Lifecycle Stages

Traditional CI/CD collapses compilation, tagging, and deployment into a chaotic single step. RROM cleanly separates them into three independent lifecycle stages with hard boundary gates.





1. The Build is the Physical Unit

A build is the tangible, immutable output of a deterministic compilation process. It occurs on ingress—the exact moment source code is merged into the integration trunk (`main`). - It produces an artifact identified by a monotonic build sequence (`:b42`), the commit SHA (`:sha-9f8a3c21e01b`), and its immutable SHA-256 digest (`sha256:7f9a...3c`). - In multi-tenant or multi-stage environments, this exact artifact is immediately attached to a moving staging stream (`:uat`). Staging exercises the exact bits that will eventually ship to production.

2. The Release is an Authorized Name

A release is pure metadata. It does not compile source code, execute bundlers, or invoke container build daemons. - When an existing build in UAT proves stable, an operator cuts a release by

giving that build a Semantic Versioning name (`v1.4.2`). - The release mechanism looks up the existing build digest in the artifact registry and adds the tag `v1.4.2` to that identical digest. - The release commit in git modifies **only** metadata files: the Helm chart version, the values digest pin, and the changelog. It is cryptographically proven to have zero source code changes relative to the build commit.

3. The Deployment is Declarative Reconciliation

A deployment is the convergence of an environment toward a declared state. - Neither developers nor CI runners push bits to production servers. - The declarative GitOps orchestrator (e.g., Argo CD on Kubernetes) observes that the configuration repository has updated its pinned image digest to `sha256:7f9a...3c` and reconciles the cluster pods to match. - Deployment completes only when post-sync runtime health checks pass on the live running system.

Why “Build on Tag” is an Anti-Pattern

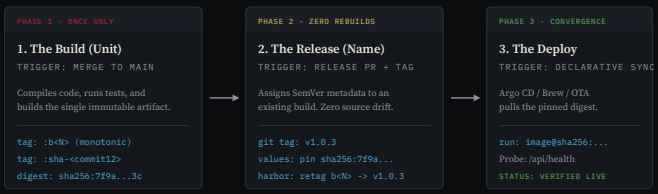
Most teams configure their CI systems to trigger container builds when a git tag (`v*`) is pushed. This pattern suffers from fatal structural flaws:

1. **Environmental Divergence:** The tagged build executes minutes, days, or weeks after the code was merged to `main`. Transient package updates in upstream registries (e.g., PyPI, npm, Alpine package mirrors) can silently alter binary behavior between staging and production.
2. **Digest Amnesia:** Because the tag creates a new build, the cryptographic digest running in production has no historical verification record in staging logs.
3. **Elevated Attack Surface:** Building artifacts at release time requires storing production compilation secrets, npm tokens, and SSH keys inside release runner environments, violating least-privilege security principles. Under RROM, the release

workflow needs only push/pull access to retag an existing registry digest.

The Core Axiom: Three Lifecycle Boundaries

BUILDS ARE THE UNIT · RELEASES ARE NAMES · DEPLOYS ARE RECONCILIATIONS



Chapter 3: The Numbered Principles

The 18 Canonical Principles

The Ravenhelm Release Operations Methodology is anchored by eighteen numbered principles (`RP-1` through `RP-18`). Every pipeline rule, script check, and operational workflow derives directly from this canon.

Category I: Artifact Custody & Immutability

`RP-1: Build Once on Ingress`

Compilation and container packaging occur exactly once, triggered by the integration event (merge to `main`). CI produces an immutable artifact identified by a cryptographic digest, an exact commit SHA, and an incremental build sequence (`:b<N>`). * *Failure Mode:* Rebuilding on release creates untracked binary deltas between staging and production.

`RP-2: Releases Never Rebuild`

The release lifecycle assigns a SemVer tag (`vX.Y.Z`) to an existing, pre-tested build digest. It never invokes a compiler, packager, bundler, or container build daemon. * *Failure Mode:* Release pipelines fail midway due to transient external mirror downtime, stalling emergency deployments.

`RP-3: Promotion by Reference`

Promoting software across environments (Dev → Staging/UAT → Prod) is strictly an act of updating declarative pointer manifests to an immutable digest. Bits never move between repositories; only metadata references change. * *Failure Mode:* Manually re-uploading artifacts across registries risks bit-rot, signature loss, and tampering.

`RP-4: Tags for Humans, Digests for Runtime`

Runtime manifests render `repository:tag@sha256:<digest>` . The tag exists solely for human readability, auditability, and CLI

ergonomics; the SHA-256 digest is the kernel truth pulled and scheduled by the runtime engine. * *Failure Mode:* Relying on mutable tags allows tag-overwriting attacks and registry caching discrepancies across cluster nodes.

`RP-5: Parity Precedes Promotion`

A release candidate must prove zero source code drift against the parent build commit (`check-release-source-parity.sh`). Release PRs touch configuration metadata (`values.yaml` , `Chart.yaml` , `CHANGELOG.md`), never application source. * *Failure Mode:* Sneaking "just one quick fix" into a release commit bypasses the build test suite and ships unvalidated code.

Category II: The Multi-Surface Rails

`RP-6: Multi-Surface Lockstep`

Repositories serving multiple surfaces (e.g., a backend service and an operator workstation CLI) must execute all applicable delivery rails within the same release cycle. A backend upgrade without matching workstation tooling is an operational outage. * *Failure Mode:* Operators using stale CLI binaries corrupt backend databases because API contracts drifted out of lockstep.

`RP-7: Sandboxed Inreplace for Ecosystem Tooling`

When packaging workstation CLIs for package managers (e.g., Homebrew) that insist on compilation from source, the formula must execute dynamic sandboxed `inreplace` of version manifests (`Cargo.toml` , `package.json`) to preserve zero-drift git history. * *Failure Mode:* Forcing version bumps into `Cargo.toml` on git branches dirties source commits and destroys release source parity.

`RP-8: Sovereign Binary Distribution`

Client desktop applications distribute through self-contained, signed archives bundled with cryptographic signatures (Minisign / Ed25519) and verified against machine-readable update manifests (`latest.json`). * *Failure Mode:* Unsigned client updates invite man-in-the-middle binary tampering and break operating system security models.

Category III: Deployment Authority & Cutover Governance

RP-9: Declarative Convergence Authority

Production servers and clusters never accept direct push deployments from developer workstations or CI scripts. An automated orchestrator (Argo CD GitOps / systemd supervisor) is the sole applying authority. * *Failure Mode:* Manual `helm upgrade` or `kubectl apply` commands executed from engineer laptops create undocumented cluster state drift.

RP-10: Merge-Then-Deploy

Approval to merge an approved release PR is approval to ship. Once merged, the release pipeline executes through completion: retagging artifacts, bumping gitops manifests, triggering deployment reconciliation, and proving live health. * *Failure Mode:* "Merge and wait" creates zombie commits in `main` that block subsequent urgent fixes.

RP-11: Cutovers Require ADRs

A deployment reconciles application code. A **cutover** changes the serving architecture (repo swap, datastore migration, transport change, breaking schema). Cutovers are never routine deployments; they require an Architectural Decision Record (ADR), an explicit data migration plan, and formal human-in-the-loop elevation. * *Failure Mode:* A team deploys an architectural replacement under a routine version bump, causing catastrophic data corruption and unrecoverable downtime (the Bifrost anti-pattern).

RP-12: Zero Ambient CI Authority

CI runners do not possess ambient cluster-admin or root credentials. Pipeline access is strictly gated via least-privilege AppRoles (OpenBao) or action-hash bound approvals (Valkyrie). * *Failure Mode:* Compromised CI runners expose long-lived root infrastructure tokens to supply chain adversaries.

Category IV: Verification & Observability

RP-13: Shell Success Is Not Proof

A command or pipeline exiting with status code `0` is an unverified claim. A deployment is proven only when the running system satisfies runtime health probes (`/api/health` , pod readiness, version query) on the live network. * *Failure Mode:* Piped commands mask shell failures, reporting "Deploy Succeeded" while container pods crash-loop in the cluster.

RP-14: Fail-Closed Rollback

Rollback is an instantaneous pointer repointing to the preceding immutable build digest in the gitops repository. Bad builds are never deleted; they simply never receive an approved production name. * *Failure Mode:* Deleting bad container tags breaks immutable audit logs and makes incident post-mortems impossible.

RP-15: Strict SemVer Contract

Breaking an existing API schema, authz tuple, ingest payload, or sentence address requires a MAJOR version bump (or documented pre-1.0 breaking changelog entry). * *Failure Mode:* Introducing backward-incompatible field removals in minor updates breaks downstream automation silently.

RP-16: Immutable Audit Ledgers

Every promotion is recorded in a tamper-evident audit ledger binding git commit SHA, build ID, registry digest, gitops PR, and the signer's identity. * *Failure Mode:* Inability to answer "who deployed what exact bits at what time" during post-incident compliance reviews.

RP-17: Separation of Blast Radii

Application code deploys are autonomous (merge → deploy). Infrastructure-plane and identity-plane mutations (Keycloak, OpenBao, Harbor RBAC) are elevation events gated by single-use cryptographic approvals bound to the plan hash. * *Failure Mode:* Treating identity configuration like application code allows rogue PRs to grant ambient admin privileges.

RP-18: Clean Worktree Custody

All release engineering and git operations execute from dedicated, disposable git worktrees conforming to the Checkout Custody Standard. Operators never rely on `git stash` across release branches.

* *Failure Mode:* Stashing changes across branch switches drops release commits, causing ghost diffs in production.

Chapter 4: The Operator Roles

Sovereign Human and Agent Responsibilities

In traditional organizations, release responsibilities are muddled across developers, DevOps engineers, and product managers. When a deployment fails, accountability is diluted. Under RROM, release management is partitioned into three distinct, non-overlapping roles:

```
[ Engineer / Agent ] ──> [ Release Custodian ] ──>
[ Delivery Closer ] ──> [ Gatekeeper ]
      Authored Diff            Prepares Release
Merges & Ships        Elevates Cutover
```

---

1. The Author (Engineer or Autonomous Coding Agent)

The Author produces the application feature, bug fix, or refactor. - **Boundaries:** The Author works within a disposable git worktree on an isolated branch. - **Deliverable:** A clean pull request containing application source changes, unit tests, and documentation updates. - **Ceiling:** The Author never tags a release, never edits production Helm values digests, and never pushes directly to `main`.

2. The Release Custodian

The Release Custodian is the guardian of artifact provenance and release metadata. This role may be fulfilled by a human technical lead or an automated release daemon. - **Responsibilities:** 1. Inspects the integration stream on `main` and identifies the target build (`:b<N>`). 2. Verifies that the build has passed automated test suites in the UAT/staging environment. 3. Executes `scripts/release-pr.sh b<N> vX.Y.Z` to generate the release pull request. 4. Enforces the **Release Parity Rule**: verifies that the release PR modifies *only* chart configurations, changelogs, and

digest pins—with zero diff in application source. 5. Formulates the annotated git tag pointing to the build's underlying commit SHA.

3. The Delivery Closer (The Supervisor)

The Delivery Closer is the sole authority empowered to approve and execute production release mergers. - **The Standing Directive:** Approval to merge a release PR is approval to ship that merge to production. The Delivery Closer does not merge a PR and walk away; they shepherd the release through artifact retagging, gitops repo updates, and live health verification. - **Single-Handed Execution:** The session that receives human authorization to ship must personally execute the merge. This responsibility cannot be delegated to subordinate workers, ensuring unbroken continuity of custody. - **Proof of Life:** The Delivery Closer terminates their session only when runtime health probes confirm the new build digest is actively serving live production traffic.

4. The Gatekeeper (Valkyrie Human-in-the-Loop)

Certain operational events carry catastrophic blast radii. These are designated as **Elevation Events**: - Modifying identity provider configurations (Keycloak/Zitadel). - Altering vault policies or secrets engine mounts (OpenBao). - Modifying artifact registry access rules (Harbor RBAC). - Executing an architectural cutover (database migration, queue swap).

For elevation events, the Gatekeeper role activates: - The CI pipeline executes `terraform plan` or equivalent validation in read-only mode and computes a cryptographic SHA-256 hash of the generated plan. - The pipeline pauses, issuing a structured Multi-Factor Action Approval (MFAA) challenge to the authorized human Gatekeeper. - The Gatekeeper inspects the plan and signs the approval challenge. - The CI pipeline verifies that the approval token matches the exact SHA-256 hash of the plan, applies the elevation, and immediately consumes the single-use token.

Chapter 5: The Artifact Surface

Registry Architecture and Digest Addressing

A container or binary registry is not a passive file dump; it is the physical custody vault of your compiled software. RROM defines strict specifications for registry configuration, naming schemes, and digest handling.

---

1. Triple-Tagging on Ingress

When code merges into `main`, the build runner compiles the software and tags the resulting image digest under three distinct references:

```
harbor.ravenhelm.dev/platform/vitki:b42
<- Monotonic Build ID
harbor.ravenhelm.dev/platform/vitki:sha-9f8a3c21e01b    <-
Source Commit SHA (12 chars)
harbor.ravenhelm.dev/platform/vitki:uat                 <-
Moving Staging Stream Tag
```

All three tags point to the **exact same cryptographic digest** (`sha256:7f9a...3c`).

Why Three Tags?

- `:b<N>` **(The Monotonic Build):** Humans and CI logs refer to builds sequentially ("Build 42 is ready for release"). Build numbers never decrement and provide instant temporal sorting.
- `:sha-<sha12>` **(Commit Linkage):** Provides instantaneous, unambiguous linkage between the binary image in the registry and the source tree in git. If git bisect isolates a bug to commit `9f8a3c21e01b`, the operator can immediately locate the artifact without searching build logs.
- `:uat` **(Moving Consumer Stream):** The staging/UAT environment tracks this moving tag. When developers push to `main`, UAT updates automatically without manual intervention.

2. Retagging by Digest (The Release Mechanism)

When cutting a release (`v1.4.2`), RROM does not rebuild the container. Instead, it instructs the registry to add a new human-readable tag to the proven build digest.

The Registry v2 API Rule

Many enterprise container registries (such as Harbor) enforce strict RBAC models where automation robot tokens are granted push and pull privileges, but denied access to administrative management APIs.

RROM scripts (`scripts/harbor-artifact.sh`) interact directly with the standard **Docker Registry HTTP API v2:** 1. Fetches the image manifest for `:b<N>` or `:sha-<commit12>` . 2. Extracts the canonical digest (`Docker-Content-Digest` header). 3. Pushes that exact manifest back to the registry under the new tag `refs/tags/vX.Y.Z` . 4. Executes a follow-up HEAD request to verify that `vX.Y.Z` resolves to the identical SHA-256 digest.

Because this operation speaks the pure distribution protocol rather than vendor-specific administrative APIs, it operates with minimal privileges and requires no Docker daemon on the release runner.

3. The Runtime Pinning Standard

In Kubernetes Helm charts and Docker Compose manifests, referencing images merely by tag (`image: vitki:v1.4.2`) is an operational vulnerability. An attacker or rogue script could overwrite the tag, or a cluster node could fail to invalidate its local image cache.

RROM mandates that all production deployment manifests render the full immutable digest:

```
# chart/values.yaml
image:
  repository: harbor.ravenhelm.dev/platform/vitki
  tag: v1.4.2
  digest:
sha256:7f9a8b1c2d3e4f5a6b7c8d9e0f1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a
  build: b42
```

In the Helm deployment template, the container image renders as:

```
image: "{{ .Values.image.repository }}:
{{ .Values.image.tag }}@{{ .Values.image.digest }}"
```

- The **tag** (`v1.4.2`) satisfies human inspection (`kubectl describe pod` immediately shows the version).
- The **digest** (`@sha256:...`) guarantees that the container runtime pulls and executes the exact verified bitstream, rejecting any mutated image layers.

Chapter 6: The Three Physical Rails

Multi-Surface Delivery Architecture

Modern engineering ecosystems rarely consist of web backends alone. A robust platform typically spans: 1. **Server-Side Backends** running on Kubernetes or bare-metal container clusters. 2. **Developer Workstation Tooling** (CLIs, Ratatui TUIs, diagnostics) run locally by engineers. 3. **Desktop Applications** (GUI client shells) installed on end-user machines.

When these surfaces drift apart, severe outages occur. An operator running a CLI compiled against an older protobuf definition can accidentally delete resources in a newly deployed backend.

RROM codifies three physical release rails, establishing standardized deployment patterns for each computing surface.

Rail A: Server & Kubernetes Cluster GitOps (Reference: Vitki)

Rail A governs server workloads deployed to production clusters.

```
[ Release PR Merged ] ──> [ Harbor Retags Digest vX.Y.Z ]
──> [ PR to norns-gitops ] ──> [ Argo CD Sync ]
```

Protocol:

1. **Artifact Packaging:** OCI container image and packaged OCI Helm chart stored in Harbor.
2. **Deployment Authority:** Argo CD is the sole applying authority. Workstations never execute `kubectl apply` or `helm upgrade`.
3. **Stream Separation:**
   - **UAT Stream:** Pinned to `main` branch manifests, pulling `:uat` on NodePort `30109`.
   - **Production Stream:** Pinned to tagged SemVer releases (`vX.Y.Z`), rendering the immutable `@sha256:` digest on NodePort `30108`.

4. **Promotion:** Cutting a production release opens a pull request in the dedicated GitOps configuration repository (`nate/norns-gitops`), updating `targetRevision: vX.Y.Z` . Argo CD detects the git commit and reconciles cluster pods.

Rail B: Workstation Developer CLI/TUI (Reference: Ullr / Yager)

Rail B governs CLI and TUI tools distributed to developer workstations via macOS Homebrew or Linux package managers.

The Problem: Homebrew Compilation vs. Source Parity

Homebrew formulas for compiled languages (Rust/Cargo, Go) compile software from source inside a sandboxed environment on the developer's machine. However, tools like Cargo require the version to be declared inside `Cargo.toml` .

If a release engineer bumps `Cargo.toml` on a release branch, that commit creates a source code delta, destroying the release parity check (`scripts/check-release-source-parity.sh`) against the parent build.

The Solution: Sandboxed Formula Inreplace

Under RROM, `Cargo.toml` in the application repository remains locked to its pre-release development version. The version bump occurs dynamically inside the Homebrew formula sandbox during the `brew install` execution:

```
# Formula/yager.rb in private tap (nate/tools)
class Yager < Formula
  desc "Terminal User Interface and agent harness"
  homepage "https://github.com/nwalker85/yager"
  url "https://github.com/nwalker85/yager/archive/refs/
tags/v0.1.33.tar.gz"
  sha256
"e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855"
  license "Proprietary"

  depends_on "rust" => :build
```

```
  def install
    # Sandboxed inreplace preserves source git parity in
the main repository
    inreplace "Cargo.toml", /^version = .*/, "version =
\"#{version}\""
    inreplace "Cargo.lock", /name = "yager"\nversion =
"[0-9.]+"/, "name = \"yager\"\nversion = \"#{version}\""

    system "cargo", "install", *std_cargo_args
  end

  test do
    assert_match "yager #{version}", shell_output("#{bin}/
yager --version")
  end
end
```

By performing the inreplace within the isolated build sandbox, the git commit history remains pristine, and the workstation binary compiles with the exact release version string.

Rail C: Client Desktop Application OTA (Reference: Idunn)

Rail C governs desktop GUI applications (e.g., Tauri v2 / Svelte / Electron) running on macOS and Windows workstations.

Protocol:

1. **Pre-Built Bundles:** Unlike Homebrew, desktop users do not compile from source. The CI ingress runner packages compressed application bundles (`.app.tar.gz` , `.dmg`).
2. **Cryptographic Signing (Minisign Ed25519):**
   - The release workflow retrieves the pre-built `:b<N>` artifact.
   - Signs the archive using the repository's dedicated Ed25519 Minisign private key (`minisign -Sm <App>.app.tar.gz -s ~/.minisign/tauri.key`).
3. **Update Manifest Generation (`latest.json`):**

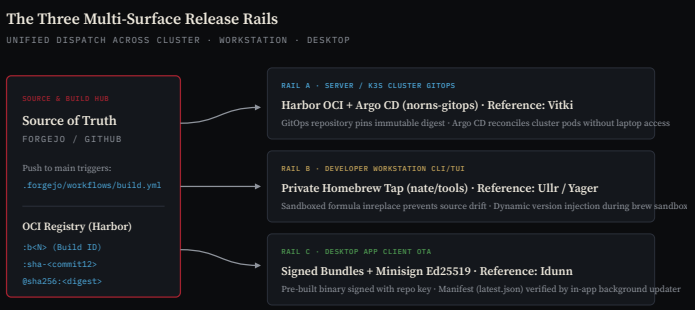
- Generates a signed update manifest containing the new version, release notes, download URL, and base64 signature.

4. **Publishing & In-App Handshake:**
   - The signed bundle and `latest.json` are attached to the GitHub/Forgejo Release.
   - The in-app background updater queries `latest.json` over HTTPS, validates the cryptographic signature using the embedded public key, downloads the payload, verifies checksums, swaps the application binary on disk, and prompts the user to restart.

Multi-Surface Dual-Rail Doctrine

When a repository ships both a server workload (Rail A) and an administrative CLI (Rail B)—such as the Ullr/Yager platform—**both rails must execute in lockstep**.

A release PR must update both `chart/values.yaml` and the Homebrew tap formula PR. Neither surface may be deployed in isolation. This ensures that the developer CLI on Nate's laptop and the container pod on the K3s cluster remain synchronized to the same underlying protocol revision.



Chapter 7: Workflow Canon & Validation Gate

The Release State Machine

The movement of software from code commit to production traffic is modeled as a 5-state finite state machine. Transitions between states are fail-closed: if any verification gate fails, the state machine halts immediately without promoting the release.

```
[ INGRESS ] ──(Build CI)──> [ CANDIDATE ] ──(Release
PR)──> [ PINNED ] ──(Tag & Gate)──> [ PROMOTED ]
──(Sync)──> [ CONVERGED ]
```

---

1. INGRESS: The Feature Merge

- **Event:** A feature or bug-fix pull request is reviewed and merged into `main`.
- **Action:** The CI workflow (`.forgejo/workflows/build.yml` or `.github/workflows/build.yml`) compiles the code, executes test suites, packages the container/binary, and pushes it to Harbor.
- **State Output:** An immutable build artifact tagged `:b<N>`, `:sha-<commit12>`, and `:uat`, carrying an immutable SHA-256 digest.

---

2. CANDIDATE: The Staging Exercise

- **Event:** The artifact is deployed automatically to the UAT / staging environment.
- **Action:** Automated integration tests, end-to-end user flows, and canary observations execute against the candidate build.
- **State Output:** A candidate build proven stable under operational load, identified by its build ID (e.g., `b42`).

3. PINNED: The Release Pull Request

- **Event:** The Release Custodian initiates the release for build `b42` targeting SemVer `v1.4.2`:

```
scripts/release-pr.sh b42 v1.4.2
```

- **Action:** The script connects to the registry, retrieves the digest for `b42`, and generates a dedicated release branch modifying **only** metadata files:
  - Updates `chart/Chart.yaml`: sets `version: 1.4.2` and `appVersion: 1.4.2`.
  - Updates `chart/values.yaml`: pins `image.tag: v1.4.2`, `image.digest: sha256:...`, and `image.build: b42`.
  - Prepends release notes to `CHANGELOG.md`.
- **The Parity Guard:** The script executes `scripts/check-release-source-parity.sh`. It computes a diff between the release branch and the candidate commit, asserting that zero application source code files have been modified. If any `.rs`, `.go`, `.ts`, or `.py` files differ, the script aborts with exit code 1.

4. PROMOTED: The Tag and Retag Gate

- **Event:** The Delivery Closer reviews and merges the release pull request into `main`, then tags the commit:

```
git tag -a v1.4.2 <commit-sha> -m "Release v1.4.2 =
build b42"
git push origin v1.4.2
```

- **Action:** The release workflow (`release.yml`) triggers on the tag and executes the three-way parity check:

```
scripts/check-release-tag.sh
```

It validates that:

```
git tag (v1.4.2) == values.yaml image.tag (v1.4.2) ==
Chart.yaml appVersion (1.4.2)
```

- **Retagging:** The release runner invokes `scripts/harbor-artifact.sh retag b42 v1.4.2`. Harbor adds the tag `v1.4.2` to the exact digest that ran in staging.

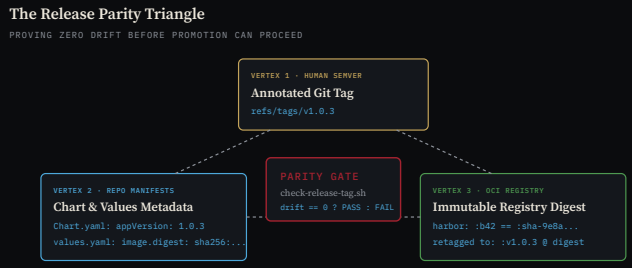
---

5. CONVERGED: GitOps Reconciliation and Proof

- **Event:** The release pipeline opens a pull request in the central GitOps repository (`nate/norns-gitops`), bumping `targetRevision: v1.4.2`.
- **Action:** Argo CD detects the updated manifest, pulls the digest `@sha256:...`, and performs a rolling update of the cluster pods.
- **Proof of Life:** The Delivery Closer probes the live running system:

```
kubectl exec -it <pod> -- node -e 'fetch("http://
localhost:3000/api/health").then(r =>
r.json()).then(console.log)'
```

- **Completion:** The release state transitions to **CONVERGED**.



Chapter 8: The Tiered Adoption Ladder

Four Maturity Tiers of Release Operations

Just as RLOM defines an honest tiering ladder for Linear workspaces, RROM structures release operations across four maturity tiers. Teams adopt RROM incrementally, climbing the ladder only when their organizational complexity and blast radius require it.

```
[ Tier 1: Artifact Discipline ] ──> [ Tier 2: Digest Locking ] ──> [ Tier 3: Multi-Rail GitOps ] ──> [ Tier 4: Sovereign Elevation ]
```

Tier 1: Artifact Discipline (The Solo Developer / Free Tier)

Focus: Stop rebuilding code on release. - **Prerequisites:** GitHub or Forgejo with standard CI runners, basic container registry (Docker Hub, GitHub Packages). - **Core Practices:** 1. Pipeline split: CI builds on push to `main` only. Builds are labeled with sequential build numbers (`b<N>`). 2. The release workflow is restricted to tagging and retagging existing container images. Zero compilation at tag time. 3. Basic tag verification: a single shell script verifies that `git tag` matches the application version string. - **Value Unlocked:** Eliminates the classic defect where code verified in staging breaks when rebuilt for production.

Tier 2: Registry & Digest Locking (The Growing Startup)

Focus: Eliminate tag-mutation risks and enforce release parity. - **Prerequisites:** Self-hosted or managed OCI registry (Harbor, AWS ECR), structured Helm chart or Compose stack.

- **Core Practices:** 1. The **Triple-Tagging Scheme**: Ingress builds publish `:b<N>`, `:sha-<commit12>`, and `:uat`. 2. **Mandatory Digest Pinning:** Production deployment manifests must pin `repository:tag@sha256:<digest>`. 3. **Automated Release Preparation:** Adoption of `scripts/release-pr.sh` and `check-release-source-parity.sh`. Release PRs are strictly verified to contain zero source code drift. 4. Registry v2 API automation: CI robot tokens are restricted to push/pull scopes, manipulating tags strictly through distribution manifests. - **Value Unlocked:** Protection against image poisoning, tag overwriting, and silent dependency shifts.

Tier 3: Multi-Rail Declarative GitOps (The Scaling Team)

Focus: Synchronize multi-surface deployments and establish GitOps authority. - **Prerequisites:** Kubernetes cluster with Argo CD / Flux, private Homebrew tap, separate GitOps configuration repository (`norns-gitops`). - **Core Practices:** 1. **Strict GitOps Boundary:** Workstation developers and CI runners are stripped of cluster apply permissions. Argo CD is the sole deployment authority. 2. **Dual-Rail Synchronization:** Repositories shipping backends and developer CLIs (e.g., Ullr/Yager) execute Rail A (K3s GitOps) and Rail B (Homebrew Tap) simultaneously within every release. 3. **Formula Inreplace Rule:** Workstation package formulas execute dynamic version inreplace in build sandboxes, keeping source git history untouched. 4. **The Merge-Then-Deploy Skill:** Approval to merge a release PR is codified as an instruction to drive deployment through live health verification. - **Value Unlocked:** Zero drift between backend APIs and developer CLI tooling; total auditability of cluster state in git.

Tier 4: Sovereign Elevation & Zero-Trust Delivery (The Enterprise)

Focus: Cryptographic elevation, blast radius isolation, and immutable audit ledgers. - **Prerequisites:** OpenBao / HashiCorp Vault, Keycloak / Zitadel identity fabric, Valkyrie MFAA broker. - **Core Practices:** 1. **Blast Radius Separation:** Application deployments execute autonomously through GitOps; control plane and identity mutations (Keycloak realms, Vault policies, Harbor RBAC) are classified as high-blast-radius elevations. 2. **Action-Hash Elevation:** High-blast mutations run read-only plans in CI, generate a SHA-256 hash of the plan, and pause for human-in-the-loop MFAA approval. 3. **Single-Use Consumable Tokens:** Approval tokens are cryptographically bound to the plan hash and can be consumed exactly once. 4. **Sovereign Client OTA (Rail C):** Desktop applications distribute via Minisign Ed25519 cryptographic signatures and self-hosted `latest.json` manifests. - **Value Unlocked:** Complete defense against rogue CI runner takeovers, internal supply chain compromise, and unauthorized infrastructure alterations.

Chapter 9: Cutovers vs. Deployments

Codifying the Cutover Boundary (ADR-011)

One of the most destructive traps in software operations is treating an **architectural cutover** as a routine **code deployment**.

Under RROM, an organization must maintain a strict, legalistic distinction between these two operational concepts.

Definitions: The Deploy vs. The Cutover

Dimension	Routine Deployment	Architectural Cutover
Definition	Reconciling a running service to a new build of the **same code/stack**.	Changing **what codebase, datastore, or transport** serves production.
Examples	Bumping Vitki from `v1.4.1` to `v1.4.2`; fixing a bug in an existing HTTP handler; updating CSS tokens.	Swapping PostgreSQL for ClickHouse; replacing a RabbitMQ broker with NATS JetStream; swapping repo implementations.
Authority	The Delivery Closer (routine PR approval and merge).	Explicit Architectural Decision Record (ADR) + Formal Human Sign-Off.
Data Plane	Backward-compatible schema additions (expand-contract).	Data migration runbook, parity verification, and rollback snapshots.
Old System	Replaced in-place via rolling pod updates.	Running in parallel until decommissioning protocol completes.

The Lessons of the Bifrost Incident

The necessity of this boundary was forged in estate history. In mid-2026, an agentic engineering session attempted to upgrade an internal tool gateway service (“Bifrost”). Rather than deploying an incremental update, the session substituted an entirely different

repository codebase, changed the datastore schema, and severed active transport layers in a single pull request.

Because the pull request was reviewed and merged as a "routine version bump," the deployment went live immediately. Production traffic crashed, active agent workflows were severed, and unmigrated database records were orphaned.

From that disaster emerged **ADR-011: The Cutover Doctrine**:

> ***NEVER replace a deployed service's architecture without an ADR and explicit cutover approval.***
> *A merge that changes what codebase or stack serves production—repo swap, datastore swap, transport removal—is a cutover, not a deploy. It requires: 1. An Architectural Decision Record (ADR) accepted prior to implementation. 2. A documented, dry-run tested data migration plan. 3. An explicit, dual-running verification window. 4. A formal decommissioning protocol for the replaced system.*

The Four Requirements of a Cutover

Whenever a proposed change satisfies the definition of a cutover, the release pipeline halts. The following four gates must be satisfied before traffic may shift:

1. The Decision Record (ADR)

An ADR must be committed to the repository's architecture decisions log (`docs/architecture/decisions/`), detailing: - The technical justification for replacing the serving architecture. - The rejected alternatives. - The backward-compatibility guarantees for existing client consumers.

2. The Expand-and-Contract Migration Plan

Datastore and schema cutovers must never occur in a single "big bang" release: - **Phase 1 (Expand):** Deploy schema additions. The application dual-writes to both the old and new structures while continuing to read from the old. - **Phase 2 (Backfill):** Offline background workers migrate historical data. Parity checks verify byte-level integrity between old and new stores. - **Phase 3 (Contract):**

The application switches reads to the new datastore. Once verified under peak production load, writes to the legacy datastore cease.

3. The Shadow-Traffic Verification

The new architecture must run concurrently alongside the legacy system in production for a minimum evaluation window. Ingress proxies mirror real production requests to both stacks, asserting response parity and error-rate equivalence.

4. Loud Decommissioning

The legacy system is not left running as an undocumented zombie process. A formal decommissioning PR removes the legacy manifests from the GitOps repository, archives the retired codebase, tears down unused network routes, and reclaims allocated storage.

Chapter 10: Hotfixes, Emergencies & Rollback

The Rollback Axiom

When a critical production defect occurs, the worst time to invent a recovery procedure is during the outage. Under RROM, rollbacks are deterministic, instant, and fail-closed:

> ***Rollback is pointer repointing; it is never binary deletion.***
> A bad build is never purged from the registry; it simply never receives an authorized release name. Rollback is the instantaneous repointing of the GitOps pointer to the preceding immutable build digest.

---

1. Zero-Downtime Rollback Execution

Because RROM strictly renders container images by immutable digest (`repository:tag@sha256:...`) in the central GitOps repository (`norns-gitops`), rolling back a failed production release does not require recompiling code, git reverts, or docker pushes.

The 60-Second Rollback Protocol:

1. Identify the previous stable release tag (e.g., `v1.4.1` , which pinned digest `sha256:4a8b...`).
2. Open an emergency commit in `norns-gitops` updating `clusters/norns/applications/<app>.yaml` :

```
spec:
  source:
    targetRevision: v1.4.1
```

3. Argo CD immediately detects the revision change. Because the container image digest `sha256:4a8b...` was already pulled by cluster nodes during the previous release, the cluster performs

an instantaneous rolling pod replacement without downloading external layers.
4. Verify live health: confirm `/api/health` returns status `200` on the restored digest.

Total elapsed recovery time: **under 60 seconds.**

---

2. The Hotfix Lifecycle

When a production outage requires a code fix rather than a rollback, teams frequently panic and attempt to commit directly to `main` or push hotfix tags without testing. RROM strictly forbids this practice.

Even in an active P0 incident, the hotfix must obey the **Ingress → Candidate → Pinned** state machine:

```
[ Hotfix PR on branch ] ──> [ Merge to main ] ──> [ CI Ingress: b43 (:uat) ] ──> [ release-pr.sh b43 v1.4.3 ] ──> [ Argo Sync ]
```

The Accelerated Hotfix Protocol:

1. Author the fix in a dedicated git worktree branched from `main`.
2. Open the hotfix pull request. CI runs automated regression tests.
3. Merge the hotfix PR into `main`. The ingress runner packages the fix once, pushing it as build `b43` (`:uat`).
4. Perform a 2-minute smoke test in the live UAT environment.
5. Execute `scripts/release-pr.sh b43 v1.4.3`. The script verifies zero application source drift relative to the hotfix commit and pins digest `b43` in `chart/values.yaml`.
6. Merge the release PR, push tag `v1.4.3`, and bump the GitOps repository.

Because this protocol follows the exact same mechanical path as routine releases, it introduces no novel operational variables during high-stress incident response.

---

3. Quarantining Bad Builds

What happens to the broken build (e.g., build `b42`) that caused the incident?

In undisciplined teams, engineers delete the tag or image from the container registry. Under RROM, **deleting artifacts is an operational violation**. - The registry digest `b42` remains immutable in Harbor. - The build is permanently recorded in the git history and incident post-mortem. - Because `b42` was removed from `values.yaml` and the GitOps target revision, no orchestrator will ever pull it again. - Preserving the broken build allows engineers to run offline forensic inspections, attach debuggers, and reproduce the defect in sandbox environments without affecting production.

Chapter 11: Security Blast Radius & Elevation

The Principle of Blast Radius Partitioning

Autonomous engineering systems and CI/CD pipelines require clear security boundaries. If an automation runner possesses the authority to rewrite root identity policies, a single prompt injection or compromised dependency can compromise the entire infrastructure estate.

RROM divides all operational mutations into two discrete security surfaces:

```
+-------------------------------------------------------------------------
+
| SURFACE 1: ROUTINE APPLICATION MUTATIONS (AUTONOMOUS /
LOW BLAST RADIUS)          |
| Scope:     Container images, Helm values, environment
variables, web assets.      |
| Authority: The Delivery Closer (Autonomous CI / Merge to
main is approval).       |
| Tokens:    Least-privilege registry push/pull tokens.
Zero cluster admin secrets. |
+-------------------------------------------------------------------------
+
                                        |
                                  [ HARD ISOLATION ]
                                        |
+-------------------------------------------------------------------------
+
| SURFACE 2: CONTROL PLANE & IDENTITY MUTATIONS
(ELEVATION / HIGH BLAST RADIUS)     |
| Scope:     Keycloak / Zitadel realms, OpenBao vault
policies, Harbor RBAC.        |
| Authority: Gated by Valkyrie Multi-Factor Action
Approval (MFAA).                |
| Gate:      Plan SHA-256 hash bound to single-use
consumable approval token.       |
+-------------------------------------------------------------------------
+
```

1. Zero Ambient Authority for CI Runners

Under traditional CI setups, pipelines are provisioned with long-lived static secrets: AWS admin access keys, Kubernetes cluster-admin kubeconfigs, or root vault tokens. If an attacker compromises a pipeline runner, they inherit ambient root authority over the entire cluster.

RROM enforces **Zero Ambient Authority**: - **Application Ingress Runners:** Possess only temporary, scoped tokens capable of pushing images to specific Harbor registry projects (`harbor.ravenhelm.dev/platform/*`). They have zero network access to Kubernetes API servers. - **GitOps Reconcilers:** Argo CD runs entirely inside the cluster netns. It pulls manifests from git; no inbound deploy port is exposed to the internet or CI runners.

---

2. Action-Hash Elevation (The Ehwaz Doctrine)

For Surface 2 mutations—such as altering identity provider mappings or modifying database access controls via Terraform—automation is untrusted:

> "Trust no agent with ambient authority; elevation via Valkyrie, action-hash bound, single use."
> — ***The Ehwaz Doctrine***

The Elevation Protocol:

1. **The Read-Only Plan:** The pull request triggers a CI runner authenticated with a read-only OpenBao AppRole (`tf-control-plane-plan`). The runner executes `terraform plan -out=tfplan.binary` .
2. **Cryptographic Hashing:** The runner computes the SHA-256 hash of the compiled plan:

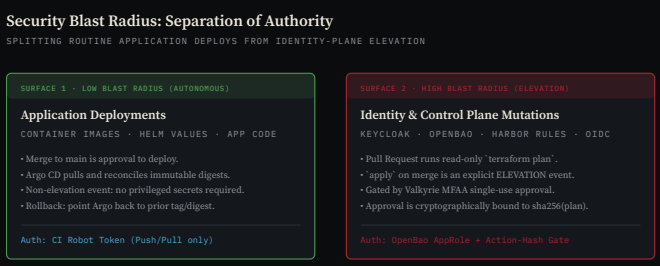
   ```
   PLAN_HASH=$(sha256sum tfplan.binary | awk '{print $1}')
   ```

3. **The Approval Challenge:** The pipeline posts an elevation challenge to the Valkyrie MFAA service:

```
{
  "action": "terraform_apply",
  "target": "identity_plane",
  "plan_hash":
"e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855",
  "requester": "ci-runner-eikthyrnir-04"
}
```

4. **Human Verification:** The authorized human Gatekeeper receives an encrypted challenge on their mobile device, inspects the exact diff, and cryptographically signs the challenge with biometric authentication.
5. **Single-Use Consumption:** The apply runner retrieves the signed authorization, verifies that the approval token's embedded `plan_hash` exactly matches the SHA-256 hash of `tfplan.binary`, executes `terraform apply`, and immediately calls the Valkyrie consume endpoint to invalidate the approval token.

Even if an attacker gains full control of the CI runner, they cannot mutate the identity plane without generating an approval token bound to the exact byte-level hash of their unauthorized plan.



Chapter 12: Code Host & Registry Integration

Engineering the Automation Substrate

RROM is agnostic to whether your source code lives on self-hosted Forgejo/Gitea, GitHub Enterprise, or GitLab. What matters is the strict enforcement of code-host hooks, runner network boundaries, and registry integration contracts.

---

1. The Build Workflow (`build.yml`)

The ingress build workflow executes on every push/merge to the default trunk (`main`). Below is the canonical workflow contract implemented in the Vitki reference:

```yaml
# .forgejo/workflows/build.yml / .github/workflows/
build.yml
name: Build & Ingress Custody

on:
  push:
    branches:
      - main
    paths-ignore:
      - 'chart/**'
      - 'docs/**'
      - '*.md'

jobs:
  build:
    runs-on: [self-hosted, eikthyrnir]
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Resolve Commit & Build ID
        id: meta
        run: |
          SHA12=$(git rev-parse --short=12 HEAD)
```

```
          BUILD_NUM="b${{ github.run_number }}"
          echo "sha=${SHA12}" >> $GITHUB_OUTPUT
          echo "build=${BUILD_NUM}" >> $GITHUB_OUTPUT

      - name: Compile and Package Image
        run: |
          podman build \
            --label
"net.ravenmask.build=${{ steps.meta.outputs.build }}" \
            --label
"org.opencontainers.image.revision=${{ steps.meta.outputs.sha }}"
\
            -t harbor.ravenhelm.dev/platform/vitki:
${{ steps.meta.outputs.build }} \
            -t harbor.ravenhelm.dev/platform/vitki:sha-
${{ steps.meta.outputs.sha }} \
            -t harbor.ravenhelm.dev/platform/vitki:uat \
            .

      - name: Push to Internal Registry
        env:
          HARBOR_USER: ${{ secrets.HARBOR_ROBOT_USER }}
          HARBOR_PASS: ${{ secrets.HARBOR_ROBOT_TOKEN }}
        run: |
          echo "$HARBOR_PASS" | podman login harbor-
nginx.lan:8443 -u "$HARBOR_USER" --password-stdin
          podman push harbor.ravenhelm.dev/platform/vitki:
${{ steps.meta.outputs.build }}
          podman push harbor.ravenhelm.dev/platform/
vitki:sha-${{ steps.meta.outputs.sha }}
          podman push harbor.ravenhelm.dev/platform/
vitki:uat
```

Critical Operational Details:

1. `paths-ignore` **Guard:** Pushes that alter only Helm charts, markdown documentation, or changelogs are explicitly ignored. This prevents the Release PR merge from triggering an unnecessary recursive build!
2. **Internal Network Bypass (** `harbor-nginx.lan` **):** CI runners push directly over high-speed local network interfaces (`10GbE / internal MTU`), bypassing external Cloudflare tunnels and upload payload size limits.

3. **OCI Labels:** The build embeds `net.ravenmask.build` and `org.opencontainers.image.revision` as immutable OCI container labels, binding the running container directly to its git commit.

---

2. The Release Workflow (`release.yml`)

When an annotated git tag (`v*`) is pushed, `release.yml` triggers. Notice that it contains **zero build commands**:

```
# .forgejo/workflows/release.yml / .github/workflows/
release.yml
name: Release & Provenance Promotion

on:
  push:
    tags:
      - 'v*'

jobs:
  verify:
    runs-on: [self-hosted, eikthyrnir]
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Enforce Parity Checks
        run: |
          scripts/check-release-tag.sh
          scripts/check-release-source-parity.sh

  promote:
    needs: verify
    runs-on: [self-hosted, eikthyrnir]
    steps:
      - name: Retag Registry Digest
        env:
          HARBOR_USER: ${{ secrets.HARBOR_ROBOT_USER }}
          HARBOR_PASS: ${{ secrets.HARBOR_ROBOT_TOKEN }}
        run: |
          # Speaks Registry v2 API: retags the build
digest to vX.Y.Z
```

```
          scripts/harbor-artifact.sh retag \
            "$(cat chart/values.yaml | yq .image.build)" \
            "${{ github.ref_name }}"

      - name: Package & Publish OCI Helm Chart
        run: |
          helm package chart/
          helm push vitki-*.tgz oci://harbor.ravenhelm.
dev/charts/
```

The release workflow is fast, light, and reliable. Because it does not compile code or pull dependencies, it completes in seconds and is completely immune to upstream registry outages.

Chapter 13: Auditing, Provenance & Compliance

Cryptographic Custody and Provenance

In high-consequence environments, compliance is not an annual questionnaire filled out for an auditor; it is the verifiable output of your release automation.

RROM mandates that every release construct a tamper-evident audit ledger that cryptographically connects: 1. **The Human / Agent Author:** The signed git commit. 2. **The Integration Record:** The CI build log and immutable build identifier (`:b<N>`). 3. **The Software Bill of Materials (SBOM):** The full inventory of upstream dependencies. 4. **The Registry Digest:** The cryptographic SHA-256 hash in Harbor. 5. **The Deployment Ledger:** The GitOps pull request and timestamped Argo CD sync status.

---

1. Automated Software Bill of Materials (SBOM)

During the single ingress build (`build.yml`), the CI runner generates a comprehensive Software Bill of Materials in standard CycloneDX or SPDX JSON format:

```
syft packages harbor.ravenhelm.dev/platform/vitki:b42 -o
cyclonedx-json=sbom.json
cosign attest --predicate sbom.json --type cyclonedx
harbor.ravenhelm.dev/platform/vitki:b42
```

The SBOM is cryptographically signed and pushed directly to Harbor as an OCI artifact attached to the container image digest.

---

2. The Verification Triangle Audit

When an internal compliance officer, auditor, or incident investigator examines a running production cluster, they execute the **Verification Triangle Audit**:

```
       [ Live Pod Image Digest ]
              /      \
             /        \
[ Harbor Tag Digest ] ── [ GitOps values.yaml Digest ]
```

1. Query the live container runtime on the cluster:

   ```
   kubectl get pod -n vitki -l app=vitki -o
   jsonpath='{.items[0].status.containerStatuses[0].imageID}'
   ```

   Output: `harbor.ravenhelm.dev/platform/vitki@sha256:7f9a...3c`

2. Query the GitOps repository configuration at tag `v1.4.2`:

   ```
   git show refs/tags/v1.4.2:chart/values.yaml |
   yq .image.digest
   ```

   Output: `sha256:7f9a...3c`

3. Query Harbor registry manifest for tag `v1.4.2`:

   ```
   crane digest harbor.ravenhelm.dev/platform/vitki:v1.4.2
   ```

   Output: `sha256:7f9a...3c`

If all three values match, **parity is proven**. There is zero possibility that an undocumented binary, unauthorized build layer, or tampered file is executing in production.

Chapter 14: The Release Maturity Assessment

Assessing Your Release Debt

Before an organization can adopt RROM, it must honestly diagnose its current operational maturity. Most engineering teams believe they have "modern CI/CD" because they use GitHub Actions or GitLab pipelines. In reality, they suffer from deep release debt.

Use this 10-point assessment rubric to grade your organization's release posture.

The 10-Point RROM Diagnostic Rubric

#	Diagnostic Question	Points (0 - 2)
1	**Build Once:** Does your release pipeline promote existing, pretested image digests, or does it trigger a compiler/rebuild at tag time? *(2 = Promotes existing; 0 = Rebuilds at tag)*	___ / 2
2	**Digest Pinning:** Do your Kubernetes manifests or Compose files pin full `@sha256:` digests, or do they reference mutable tags? *(2 = Mandatory digests; 0 = Mutable tags)*	___ / 2
3	**Staging Parity:** Is the artifact running in staging byte-identical to the artifact promoted to production? *(2 = Byte-identical; 0 = Recompiled separately)*	___ / 2
4	**Workstation Lockstep:** Are developer CLIs and admin TUIs released in lockstep with backend APIs using sandboxed formula versioning? *(2 = Synchronized lockstep; 0 = Uncoordinated)*	___ / 2
10	[illegible]	___ / 2

Scoring & Maturity Interpretation

- **0 – 8 Points (Fragile Delivery):** Your team is at extreme risk of production deployment outages, dependency drift, and supply chain tampering. Adopting **Tier 1 (Artifact Discipline)** and eliminating tag-time rebuilds should be an immediate priority.
- **10 – 14 Points (Emerging Discipline):** You have basic container discipline, but suffer from surface drift between backend APIs and developer CLIs, or allow ambient secrets in CI pipelines. Adopt **Tier 2 and Tier 3 (Multi-Rail GitOps)**.
- **16 – 20 Points (Sovereign Operations):** Your organization operates at the highest echelon of release engineering. Your releases are immutable, verified, multi-surface, and auditable.

Commercial Implementation & Advisory

Installing RROM within an existing enterprise requires careful navigation of legacy pipelines, developer habits, and compliance boundaries.

Ravenhelm LLC provides specialized executive advisory, pipeline refactoring, and multi-surface release architecture services: - **3-Day Release Pipeline Assessment:** Comprehensive audit of your repository workflows, registry topology, and cluster security boundaries. - **RROM Implementation Engagements:** Turnkey installation of the Build-Once rail, Argo CD GitOps integration, and private Homebrew formula automation. - **Valkyrie Sovereign Elevation Architecture:** Designing zero-trust action-hash elevation systems for high-consequence infrastructure.

For engagement inquiries, contact:

Nathan Walker
Founder & Principal, Ravenhelm LLC
Email: nate@ravenhelm.co
Web: https://ravenhelm.co

Appendix A: Formula Inreplace Playbook

Preserving Source Parity in Workstation Package Taps

Workstation package managers like macOS Homebrew, Arch AUR, or Alpine aports compile software directly from source tarballs on user machines.

When distributing compiled CLI tools written in Rust, Go, or Node.js, package managers often require the release version number to be declared directly inside source control manifests (`Cargo.toml` , `package.json`).

Under traditional workflows, bumping these files on a release branch creates an application source commit, violating **RP-5: Parity Precedes Promotion**.

This playbook provides concrete patterns for executing **Sandboxed Formula Inreplace (RP-7)** to dynamically inject release versioning inside package manager build sandboxes without dirtying source git commits.

---

1. Rust / Cargo Formula Pattern

In a Rust project, `Cargo.toml` in git remains pinned to its development placeholder (e.g., `version = "0.1.0"`).

In your private Homebrew tap formula (`Formula/<tool>.rb`):

```
class Tool < Formula
  desc "Sovereign operator CLI"
  homepage "https://github.com/nwalker85/tool"
  url "https://github.com/nwalker85/tool/archive/refs/
tags/v1.4.2.tar.gz"
  sha256
"7f9a8b1c2d3e4f5a6b7c8d9e0f1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a"
  license "Proprietary"
```

```
  depends_on "rust" => :build

  def install
    # Dynamically inject version into Cargo manifests
inside Homebrew's sandbox
    inreplace "Cargo.toml", /^version = .*/, "version =
\"#{version}\""
    inreplace "Cargo.lock", /name = "tool"\nversion =
"[0-9.]+"/, "name = \"tool\"\nversion = \"#{version}\""

    system "cargo", "install", *std_cargo_args
  end

  test do
    assert_match "tool #{version}", shell_output("#{bin}/
tool --version")
  end
end
```

2. Go / Ldflags Formula Pattern

Go tools can avoid modifying files entirely by injecting version metadata via linker flags (`-ldflags`):

```
class GoTool < Formula
  desc "Sovereign platform CLI"
  homepage "https://github.com/nwalker85/gotool"
  url "https://github.com/nwalker85/gotool/archive/refs/
tags/v1.4.2.tar.gz"
  sha256
"4a8b1c2d3e4f5a6b7c8d9e0f1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a"

  depends_on "go" => :build

  def install
    commit = "9f8a3c21e01b"
    ldflags = %W[
      -s -w
      -X main.Version=#{version}
      -X main.Commit=#{commit}
      -X main.BuiltBy=homebrew
    ]
```

```
    system "go", "build", *std_go_args(ldflags: ldflags),
"./cmd/gotool"
  end

  test do
    assert_match "gotool version #{version}",
shell_output("#{bin}/gotool version")
  end
end
```

---

3. Node.js / TypeScript Package Inreplace Pattern

In TypeScript CLI projects, `package.json` in git remains unbumped:

```
class NodeCli < Formula
  desc "Node-based operator utility"
  homepage "https://github.com/nwalker85/nodecli"
  url "https://github.com/nwalker85/nodecli/archive/refs/
tags/v1.4.2.tar.gz"
  sha256 "..."

  depends_on "node"

  def install
    # Dynamically rewrite package.json version field
    inreplace "package.json", /"version":\s*"[^"]+"/,
"\"version\": \"#{version}\""

    system "npm", "install",
*Language::Node.std_npm_install_args(libexec)
    bin.install_symlink Dir["#{libexec}/bin/*"]
  end
end
```

By confining all version mutations to the package manager's ephemeral build sandbox, your git history remains 100% clean, and your binary release parity checks pass every time.

Appendix B: The Vitki Reference Specification

The Canonical Reference Architecture

Vitki (`nate/vitki`) is the production reference implementation that birthed the Ravenhelm Release Operations Methodology. This appendix details its concrete directory layout, architectural decisions, and release helper scripts.

---

1. Directory Topology

```
vitki/
├── .forgejo/
│   └── workflows/
│       ├── build.yml                  <- Ingress build on
merge to main
│       └── release.yml                <- Parity
verification & Harbor retag on tag
├── chart/
│   ├── Chart.yaml                     <- SemVer metadata
(version, appVersion)
│   ├── values.yaml                    <- Production
values: pins tag, digest, build
│   ├── values-uat.yaml                <- Staging values:
follows :uat tag
│   └── templates/
│       └── deployment.yaml            <- Renders
repository:tag@digest
├── scripts/
│   ├── release-pr.sh                  <- Pins build b<N>
to vX.Y.Z in values.yaml
│   ├── harbor-artifact.sh             <- Speaks Registry
v2 API: retags digests
│   ├── check-release-tag.sh           <- Verifies tag ==
Chart == values
│   └── check-release-source-parity.sh <- Proves zero
application source drift
└── docs/
    └── architecture/
```

```
        └── decisions/
            ├── 0002-k3s-runtime-k0s-local.md
            ├── 0003-helm-harbor-argocd.md
            ├── 0004-semver-prod-tags.md
            └── 0007-builds-are-the-unit-releases-are-
names.md
```

---

2. Core Helper Scripts

`scripts/release-pr.sh` (Release Pinning)

```
#!/usr/bin/env bash
# Usage: scripts/release-pr.sh <build-id> <semver-tag>
# Example: scripts/release-pr.sh b42 v1.4.2

set -euo pipefail

BUILD="$1"
TAG="$2"
SEMVER="${TAG#v}"

# 1. Resolve digest for build b<N> from Harbor
DIGEST=$(scripts/harbor-artifact.sh resolve "${BUILD}")

# 2. Update Helm Chart metadata
yq -i ".version = \"${SEMVER}\"" chart/Chart.yaml
yq -i ".appVersion = \"${SEMVER}\"" chart/Chart.yaml

# 3. Pin image digest in values.yaml
yq -i ".image.tag = \"${TAG}\"" chart/values.yaml
yq -i ".image.digest = \"${DIGEST}\"" chart/values.yaml
yq -i ".image.build = \"${BUILD}\"" chart/values.yaml

# 4. Enforce source parity check
scripts/check-release-source-parity.sh
```

`scripts/check-release-source-parity.sh` (Parity Guard)

```
#!/usr/bin/env bash
# Asserts that the release commit modified ONLY chart and
changelog files.

set -euo pipefail

FORBIDDEN_DIFFS=$(git diff --name-only origin/main | grep
-vE '^(chart/|CHANGELOG\.md|docs/runbooks/release\.md)' ||
true)

if [ -n "${FORBIDDEN_DIFFS}" ]; then
  echo "ERROR: Release PR touches application source
files!" >&2
  echo "${FORBIDDEN_DIFFS}" >&2
  exit 1
fi

echo "PASS: Zero application source drift proven."
```

`scripts/check-release-tag.sh` (Parity Triangle)

```
#!/usr/bin/env bash
# Validates that git tag == values.yaml tag == Chart.yaml
appVersion

set -euo pipefail

GIT_TAG="${GITHUB_REF_NAME:-$(git describe --tags --exact-
match)}"
VALUES_TAG=$(yq .image.tag chart/values.yaml)
CHART_APP_VER="v$(yq .appVersion chart/Chart.yaml)"

if [ "${GIT_TAG}" != "${VALUES_TAG}" ] || [ "${GIT_TAG}" !
= "${CHART_APP_VER}" ]; then
  echo "ERROR: Release parity triangle mismatch!" >&2
  echo "git tag:       ${GIT_TAG}" >&2
  echo "values.yaml:   ${VALUES_TAG}" >&2
  echo "Chart.yaml:    ${CHART_APP_VER}" >&2
  exit 1
fi
```

```
echo "PASS: Release parity triangle verified."
```

By standardizing these exact scripts across all platform and product repositories, Ravenhelm maintains unbroken release discipline across its entire software fleet.



Ravenhelm Release Operations Methodology (RROM)
Version 1.0.0
Authored by Nathan Walker.
© 2026 Ravenhelm LLC. All rights reserved.
Published by Ravenhelm Publishing Series LLC.
Distributed under the RROM Community License 1.0.
Set in Source Serif 4 and IBM Plex Mono.
Typeset using Typst 0.15.1 on macOS.
ravenhelm.co · nwalker.cc